When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

There are a total of 18 cases that could have resulted from the audit test; however, using the old system, the failure in security was prevented in 4 of the cases. In contrast, ERMA-GRC prevented the entire 18 cases. ERMA-GRC prevented writes in a session, where in the user is unable to approve his/her record, stamping audit lines to logged-in users but not the user's name and detects edit to an entry and reports the unavailability of the database as a failure. Besides the audit test criteria mentioned above, ERMA-GRC is able to block all but 3 out of the 4 failures in total and that is in two occasions where it blocked 2 simultaneous approvals for the same record. The storage results were obtained from the simulator of the database and not the system. In two circumstances, the new design did not block the failure and they are explained below. In spite of the risk indicator showing the correct state on the screen, a client has the capacity of connecting to the server and setting the state to whatever he/she wishes. Also, an individual with access to the audit key can manipulate the audit trail offline and undetected once the server reboots. The principles are also dependent on each other in the sense that the reliability of audit depends on the separation of duties.

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. This paper provides a taxonomy of failures observed in a working system, seven principles together with two testable propositions towards a design theory, and a scenario protocol that others can rerun. They may utilize these principles in the form of actions. These two problems identified in the design lead to the three exercises as follows: external anchoring of the audit trail from the system; calculation of the entire metrics from the server, and the test of the principles with the practitioners.

Keywords

assurance integrity; GRC information systems; design science research; enterprise risk management; tamper-proof audit trail; separation of duties